Python Programming

Problem solving activity – 4

Name : Kunda Joshita

USN : 25BTRCC031

Question 1: Operator Precedence, Type Conversion and Conditional Reasoning

A. Error diagnosis

1. **No type conversion.** input().split() returns strings. The first line, m1 + m2 + m3 / 3, immediately raises Type Error (str / int is not allowed). Even if it didn't, m1 >= 40 compares a string to an integer, which also fails.
2. **Operator precedence in the average.** / binds tighter than +, so the code computes m1 + m2 + (m3/3). For 80 70 90 it gives 180.0 instead of 80.0. The sum must be parenthesized: (m1 + m2 + m3) / 3.
3. **Boolean precedence.** and binds tighter than or, so the condition is (m1>=40 and m2>=40) or (m3>=40). A student with m3 >= 40 passes even if m1 or m2 is below 40. The rule requires all three marks to be at least 40.
4. **Weaknesses.** There is no check for the number of inputs or for non-numeric input. The wrong average also makes the average >= 75 test almost always true.

B. Corrected code

```
def evaluate(m1, m2, m3, attendance):
    average = (m1 + m2 + m3) / 3
    all_marks_ok = m1 >= 40 and m2 >= 40 and m3 >= 40
    if all_marks_ok and average >= 75 and attendance >= 75:
        return average, "Eligible"
    return average, "Not Eligible"


try:
    data = input("Enter marks and attendance: ").split()
    if len(data) != 4:
        raise ValueError("Exactly four values are required")
```

```
    m1, m2, m3, attendance = [float(x) for x in data]
    result = evaluate(m1, m2, m3, attendance)
    print("Average:", round(result[0], 2))
    print("Status:", result[1])
except ValueError as e:
    print("Invalid input:", e)
```

C. Testing

Test 1 : 80 70 90 85

Output :

```
Enter marks and attendance: 80 70 90 85
Average: 80.0
Status: Eligible
```

Test 2 : 30 100 100 85

Output 2 :

```
Enter marks and attendance: 30 100 100 85
Average: 76.67
Status: Not Eligible
```

Test 3 : 75 75 75 75

Output 3 :

```
Enter marks and attendance: 75 75 75 75
Average: 75.0
Status: Eligible
```

Test 4 : 100 100 100 74

Output 4 :

```
Enter marks and attendance: 100 100 100 74
Average: 100.0
Status: Not Eligible
```

Test 5 : 80 74

Output 5 :

```
Enter marks and attendance: 80 74
Invalid input: Exactly four values are required
```

Why eligibility changes: in `30 100 100 85` the average is 76.67 and attendance is 85, so two conditions are satisfied. But 30 < 40 violates the "all subjects at least 40" rule. The corrected code uses `and` between all three marks, so one failing mark makes the whole condition false. The original `or m3 >= 40` let this student through.

D. Reflection

The hardest error to find was the `and`/`or` precedence, because the code looked as if it checked all three marks. It only became clear when I tried a student with one failing mark and saw that the condition still passed. The wrong average was easy to confirm by hand, since 80, 70, 90 should average 80, not 180. I verified the fix by hand-calculating each test case and comparing it with the program output, including boundary values (exactly 75). I learned that `/` runs before `+`, that `and` runs before `or`, and that parentheses are the safest way to show intent. I also learned that `input()` always gives strings, so conversion is the first step of any numeric program.

Question 2: Nested Loops, Loop Control and Prime Generation

A. Error diagnosis

1. **Range boundary.** range(start, end) excludes end, so end is never tested. It should be range(start, end + 1).
2. **Divisor range is too small.** range(2, int(number ** 0.5)) stops before int(sqrt). For 4, 6, 8 the range is empty (range(2,2)), so they are declared prime. For 9, 15 and 25 the divisor 3 or 5 is never tried. It must be range(2, int(number**0.5) + 1).
3. **continue instead of break.** Once a divisor is found, continue keeps testing more divisors for no reason. break stops the loop immediately.
4. **Numbers below 2.** They are flagged non-prime, but the divisor loop still runs for them. For negative numbers, number ** 0.5 becomes a complex number and int() raises TypeError. These numbers should be skipped.

B. Corrected code

```
import math

def generate_primes(start, end):
  primes = []
  for number in range(start, end + 1):
    if number < 2:
      continue              # never prime
    is_prime = True
```

```
    for divisor in range(2, math.isqrt(number) + 1):
      if number % divisor == 0:
        is_prime = False
        break              # stop at first divisor
    if is_prime:
      primes.append(number)
  return primes

print(generate_primes(1, 30))
print(generate_primes(2, 2))
print(generate_primes(14, 16))
```

C. Testing

Output :

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
[2]
[]
```

D. Reflection

A composite number n has a factor no larger than √n, so I only need to test divisors from 2 up to √n **inclusive**. The word "inclusive" matters: perfect squares like 9 and 25 have √n as their only divisor, which the original range skipped. I found this by checking my output against the primes I know and noticing 9 and 25 in the list. The case (2, 2) exposed the off-by-one error in the outer range. (14, 16) confirmed that composite numbers are no longer misclassified. Loop control matters because break prevents wasted work, while continue only skips to the next iteration of the same loop, which is a different effect. Good tests include squares of primes, boundary values and an empty result.

Question 3: Recursion, Base Cases and Return Values

A. Error diagnosis

The recursive call `digit_sum(number // 10)` is made, but its return value is thrown away. The function then returns only the last digit.

Trace of `digit_sum(5824)`:

- `digit_sum(5824)`: digit = 4, calls `digit_sum(582)` (result ignored), returns **4**

- The calls for 582 (returning 2), 58 (returning 8) and 5 (returning 5) all run, but their results are discarded.

So the program prints `Sum of digits: 4` instead of 19. A recursive call only computes a value. If you don't use it, it contributes nothing.

B. Corrected code

```
def digit_sum(number):
  number = abs(number)       # handles negative input safely
  if number < 10:            # base case: single digit (also covers 0)
    return number
  return number % 10 + digit_sum(number // 10)
number = 5824
print("Sum of digits:", digit_sum(number))
```

C. Testing

Output :

```
Sum of digits: 19
```

D. Reflection

Making a recursive call only runs the function again. Using its return value, as in `digit + digit_sum(...)`, is what carries the answer back up the call chain. In my first trace of the original code I saw that the calls for 582, 58 and 5 returned correct values, but nobody used them. The base case `number < 10` stops the recursion and supplies the first real value, which then gets added at each level on the way back. Writing the stack of calls on paper made this clear. Testing 0 and 7 confirmed that the base case handles both without recursing.

Question 4: Lists, Mutation, Comprehensions and Sorting

A. Error diagnosis

1. **Modifying a list while iterating.** marks.remove(mark) shifts elements left, and the loop's internal index moves on, so the element after each removed item is skipped. In this data, 60 after the first 35 and 55 after the second 35 are skipped. It happens to work here, but two adjacent bad marks like [35, 35] would leave one behind.
2. **Wrong qualification condition.** mark > 60 excludes exactly 60, but the requirement is >= 60.
3. **Wrong sort order.** sort(reverse=True) gives descending order, but requirement 3 says ascending.

4. **Duplicates and top-three logic.** set() removes duplicates but loses order, so sorting must happen afterwards. Taking [:3] only works because of the descending sort, which contradicts requirement 3.

B. Corrected code

```
marks = [35, 60, 45, 80, 35, 55, 60, 95, 40, 80]

# 1 + 2 + 3: filter (>=40), remove duplicates (set), sort ascending
unique_marks = sorted({m for m in marks if m >= 40})

# 4: qualifying marks (>=60), still ascending
qualified = [m for m in unique_marks if m >= 60]

# 5: three highest (all if fewer than three), highest first
top_three = sorted(qualified, reverse=True)[:3]

print("Processed marks:", unique_marks)
print("Qualified marks:", qualified)
print("Top three:", top_three)
```

C. Testing

Output :

```
Processed marks: [40, 45, 55, 60, 80, 95]
Qualified marks: [60, 80, 95]
Top three: [95, 80, 60]
```

D. Reflection

Removing items while looping over the same list confuses the loop because the list shrinks while the iterator still counts positions. Neighboring items slide into the removed item's position and get skipped. I understood this by tracing index by index. The safer approach is to build a new list with comprehension and leave the original untouched. To verify the order, I printed the lists and checked that each value was larger than the previous one. For boundaries, I included 40 and 60 deliberately and confirmed they were kept. I learned that > and >= are different, and that sorting must happen after de-duplication because sets don't preserve order.

Question 5: Dictionaries, Sets, Nested Data and Consistency

A. Error diagnosis

1. **Wrong threshold.** len(common) > 2 requires three or more shared courses, but "at least two" needs >= 2. Bala and Divya, who share exactly two, are wrongly excluded.
2. **Duplicates.** Arun's list has "Python" twice and Ezhil's has "Java" twice. Converting to sets handles this, but counting list items would inflate the counts. Sets must be used (or distinctness otherwise ensured).
3. **Unordered output.** list(common) turns a set into a list in arbitrary order, so results may differ between runs or Python versions. They need sorted().
4. **Weakness.** registrations[target] raises KeyError if the target is absent, and the set of Arun's courses is rebuilt every iteration.

B. Corrected code

```
registrations = {
  "Arun": ["Python", "Java", "SQL", "Python"],
  "Bala": ["Python", "C", "SQL"],
  "Charan": ["Java", "Python", "SQL"],
  "Divya": ["Python", "Java"],
  "Ezhil": ["C", "Java", "Java"]
}

target = "Arun"
target_courses = set(registrations.get(target, []))  # distinct courses, built once
common_students = {}

for student, courses in registrations.items():
  if student != target:
    common = target_courses & set(courses)
    if len(common) >= 2:
      common_students[student] = sorted(common)

print("Common courses:", common_students)
```

C.Testing

Output :

```
Common courses: {'Bala': ['Python', 'SQL'], 'Charan': ['Java', 'Python', 'SQL'], 'Divya': ['Java', 'Python']}
```

D. Reflection

A set is the natural tool for “what do these two groups have in common”, because the & operator gives the intersection directly and automatically ignores repeated entries. This solved the duplicate problem for Arun and Ezhil without extra code. I found the threshold bug by computing the expected answer by hand first (Bala, Charan and Divya) and noticing that the original printed only Charan. Sets have no guaranteed order, so I used sorted() to make the output the same on every run, which also makes it easy to compare with the expected answer.